Chapter 11 — Lambdas and Functional Programming

Until Java 8, if you wanted to hand a piece of *behaviour* to a method — "sort by length", "run this later", "keep only the big orders" — you had to wrap it in an object, usually an anonymous class with five lines of ceremony around one line of logic. **Lambdas** (Java 8) let you write that behaviour directly as a value: `s -> s.length()`. Together with **functional interfaces**, **method references** and `Optional`, they brought a **functional style** (programming mostly by passing and combining functions, rather than by changing variables step by step) into Java that the collections API (Chapter 10) and streams (Chapter 12) are built on.

This chapter explains how lambdas work, which built-in function types exist and when to use each, how to combine small functions into bigger ones, how to use `Optional` well (and badly), and which functional-programming ideas pay off in everyday Java.

What you'll learn

1. Why lambdas exist: passing behaviour as a parameter.
2. Functional interfaces, lambda syntax and target typing; how lambdas differ from anonymous classes.
3. The `java.util.function` catalogue, including primitive specialisations.
4. The four kinds of method reference.
5. Variable capture and "effectively final"; what `this` means inside a lambda.
6. Composing functions: `andThen`, `compose`, `Predicate.and/or/not`, function factories.
7. `Optional`: the right uses, the wrong uses.
8. Functional style in practice: pure functions, immutability, higher-order functions, currying, laziness, memoisation, strategies, checked exceptions.

All runnable code is in `code/` (package `ch11.*`), compiled with `javac --release 25` and run on JDK 26 (the **JDK**, Java Development Kit, is the toolset that compiles and runs Java; `javac` is its compiler, and `--release 25` tells it to target the Java 25 language and library level).

> **Exam depth.** Which lambda forms compile, `var` in parameters, method-reference puzzles, the full built-in interface table with method names (`test`, `apply`, `accept`, `get`, `applyAsInt`…) and "effectively final" traps are drilled in OCP Chapter 8 — Lambdas and functional interfaces (see *Gotchas & exam traps*). `Optional` exam questions are in OCP Chapter 10.

11.1 Why lambdas: passing behaviour

The problem it solves

Many methods need a small piece of logic from the caller: `sort` needs "how to compare", a button needs "what to do when clicked", a thread needs "what to run", a filter needs "what to keep". This is called **behaviour parameterisation** — making behaviour an argument, just like data.

Before Java 8 the only way was an object implementing an interface, typically an **anonymous class** (a class defined and instantiated in one expression, §6.11):

```
names.sort(new Comparator<String>() {
    @Override public int compare(String a, String b) { return Integer.compare(a.length(), b.length()); }
});
```

Five lines, of which only `Integer.compare(a.length(), b.length())` is the idea. A **lambda expression** is an anonymous function: parameters, an arrow, a body.

```
names.sort((a, b) -> Integer.compare(a.length(), b.length()));
```

Analogy: instead of hiring a whole employee (a class) to do one task, you hand over a sticky note with the instructions (the lambda). The receiving method decides when — and whether — to follow them.

Why it matters

- **Less noise**, more intent — *Effective Java* (Joshua Bloch's widely used book of numbered Java best-practice "Items") Item 42: "prefer lambdas to anonymous classes".
- APIs could become far more flexible: `Map.merge`, `List.removeIf`, `Comparator.comparing`, `CompletableFuture`, the Streams API — all take functions.
- Deferred and conditional execution (code that runs later, or only if some condition holds): pass a `Supplier` and the work happens only if needed (§11.8).

11.2 Functional interfaces and lambda syntax

What it is

A **functional interface** is an interface with **exactly one abstract method** (a method declared with no body, which an implementing class or lambda must supply) (a "single abstract method", or **SAM**, type). A lambda is simply a compact way to create an instance of one. `Runnable` (`void run()`), `Comparator<T>` (`int compare(T, T)`) and `Callable<V>` are classic examples.

Rules:

1. Only *abstract* methods count. `default` and `static` methods are fine, and so are abstract re-declarations of `Object`'s public methods (that's why `Comparator` can declare `equals` and still be functional).
2. The optional `@FunctionalInterface` **annotation** (an `@Name` tag attached to code that the compiler or tools can read) asks the compiler to verify the "exactly one" rule — a guard against someone later adding a second abstract method.
3. A lambda has **no type of its own**. Its type comes from the context it appears in (the compiler works it out, which is called **type inference**) — the **target type** (an assignment, a method parameter, a return, a cast). So the same text `s -> s.isEmpty()` can become a `Predicate<String>` or a `Function<String, Boolean>`. That's also why `var f = x -> x;` doesn't compile: there is no target type.

Syntax

Form	Example	Notes
No parameters	`() -> System.out.println("hi")`	parentheses required
One parameter, inferred type	`s -> s.length()`	parentheses optional
Several parameters	`(a, b) -> a + b`	parentheses required
Explicit types	`(Integer a, Integer b) -> a + b`	all or none typed
`var` parameters (Java 11)	`(var a, var b) -> a * b`	lets you add annotations: `(@NonNull var s) -> …`; all or none
Unnamed parameter (Java 22)	`(key, _) -> key`	`_` for a parameter you don't use
Expression body	`x -> x * 2`	value of the expression is returned
Block body	`n -> { if (n < 0) return "neg"; return "ok"; }`	braces, semicolons and `return` required

Code

```java
// code/ch11/lambdas/LambdaBasics.java (key parts)
@FunctionalInterface
interface Discount {
    double apply(double price);                                    // the single abstract method
    default Discount then(Discount next) { return p -> next.apply(apply(p)); }   // defaults allowed
}

Runnable noArgs                 = () -> System.out.println("no parameters");
Function<String, Integer> oneArg = s -> s.length();
BinaryOperator<Integer> typed    = (Integer a, Integer b) -> a + b;
BiFunction<Integer, Integer, Integer> withVar = (var a, var b) -> a * b;
Function<Integer, String> block  = n -> { if (n < 0) return "negative"; return "non-negative"; };

Predicate<String> isEmpty1          = s -> s.isEmpty();   // same text,
Function<String, Boolean> isEmpty2  = s -> s.isEmpty();   // different target types

Discount tenOff = p -> p - 10, twentyPercent = p -> p * 0.8;
tenOff.then(twentyPercent).apply(100);                    // (100 - 10) * 0.8

Map<String, BinaryOperator<Integer>> ops = Map.of("+", Integer::sum, "*", (a, b) -> a * b, "max", Math::max);
```

Output:

```
anonymous class: [al, Bea, Charlie]
lambda:          [Charlie, Bea, al]
no parameters
6 5 20 negative
Predicate.test=true Function.apply=false
100 -> tenOff then 20% = 72.0
+=13 *=42 max=7
```

The last line shows lambdas are *values*: stored in a map, looked up, called — a calculator's operator table in one line.

How it works under the hood

1. The compiler turns the lambda body into a private method of the enclosing class (named like `lambda$main$0`).
2. At the lambda's location it emits an `invokedynamic` instruction (a **bytecode** instruction — bytecode being the compact, portable instruction format that `javac` produces and the JVM runs — whose target is decided at run time on first use instead of being fixed at compile time). The first time it runs, the JVM's (Java Virtual Machine: the program that loads and executes Java bytecode) `LambdaMetafactory` generates a small class implementing the functional interface that calls that private method.
3. No separate `.class` file per lambda appears on disk (anonymous classes produce `Outer$1.class`).
4. A lambda that captures nothing is typically created once and reused; one that captures variables gets a new instance holding those values. (Don't rely on identity either way — it's unspecified.)

Lambda vs anonymous class

	Lambda	Anonymous class
Implements	only functional interfaces	any interface or class (can extend an abstract class)
Number of methods	one	any; can have fields and state
`this` refers to	the **enclosing** instance	the anonymous object itself
Shadowing (a new variable hiding an outer one with the same name)	can't redeclare a local variable of the enclosing scope	can shadow
Compiled as	`invokedynamic` + private method	a separate class file

Use an anonymous class only when you need state, several methods, or to extend a class.

Common mistakes

- A block body that forgets `return`, or an expression body followed by a semicolon inside: `x -> { x * 2 }` ✗ vs `x -> x * 2` ✓.
- Mixing typed and untyped parameters: `(String a, b) -> …` ✗.
- Naming a lambda parameter the same as an existing local variable → "variable is already defined".
- Long multi-statement lambdas — extract a named method and use a method reference; stack traces (the list of method calls printed when an exception is thrown) and readers both benefit.

Self-check

1. What makes an interface "functional"? → Exactly one abstract method (Object's public methods and defaults/statics don't count).
2. Why can't you write `var f = () -> 42;`? → A lambda needs a target type; `var` provides none.
3. What does `this` mean inside a lambda? → The enclosing object, same as outside the lambda.

TL;DR

A lambda is an anonymous function that implements a functional interface (one abstract method). Its type comes from context. Syntax: `params -> expression` or `params -> { statements; }`. Prefer it over anonymous classes unless you need state or `this` to mean the object itself.

11.3 The built-in functional interfaces (`java.util.function`)

What it is

You rarely need to write your own functional interface: `java.util.function` has **43** of them covering the common shapes. *Effective Java* Item 44: favour the standard ones — callers already know them and they come with useful **default methods** (interface methods that have a body, so every implementation gets them for free).

The core shapes

Interface	Shape	Method	Typical use
`Supplier<T>`	`() → T`	`get()`	factories, lazy values, defaults
`Consumer<T>`	`T → void`	`accept(t)`	`forEach`, logging, sending
`BiConsumer<T,U>`	`(T, U) → void`	`accept(t, u)`	`Map.forEach`
`Predicate<T>`	`T → boolean`	`test(t)`	`filter`, `removeIf`
`BiPredicate<T,U>`	`(T, U) → boolean`	`test(t, u)`	two-argument checks
`Function<T,R>`	`T → R`	`apply(t)`	`map`, key extractors
`BiFunction<T,U,R>`	`(T, U) → R`	`apply(t, u)`	`Map.merge`, `compute`
`UnaryOperator<T>`	`T → T`	`apply(t)`	`List.replaceAll`, transformations
`BinaryOperator<T>`	`(T, T) → T`	`apply(a, b)`	`reduce`, `merge` functions

Older JDK types are functional interfaces too: `Runnable` (`() → void`), `Callable<V>` (`() → V`, may throw), `Comparator<T>`.

Primitive specialisations

Generics (type parameters like `<T>`, Chapter 9) only work with objects, so a `Function<Integer, Integer>` **boxes** every `int` into an `Integer` (wraps the plain number in a heap object; *unboxing* is the reverse)

(allocating objects outside the small `Integer` cache — the JDK keeps ready-made `Integer` objects for -128..127 — Chapter 7). The primitive versions avoid that. Their names follow a pattern:

Pattern	Meaning	Examples
`IntX`, `LongX`, `DoubleX`	the **input** is that primitive	`IntPredicate`, `IntFunction<R>`, `IntConsumer`, `IntUnaryOperator`, `IntBinaryOperator`
`ToIntX`, `ToLongX`, `ToDoubleX`	the **output** is that primitive	`ToIntFunction<T>`, `ToDoubleBiFunction<T,U>`
`IntToLongFunction`, `DoubleToIntFunction`, …	primitive to primitive	
`XSupplier`	supplies a primitive	`IntSupplier` (`getAsInt`), `BooleanSupplier` (`getAsBoolean`)
`ObjIntConsumer<T>` etc.	object + primitive, no result	`(s, n) -> …`

The method names change accordingly: `applyAsInt`, `applyAsDouble`, `getAsInt`, `test`, `accept`. Only `int`, `long` and `double` get full sets (plus `BooleanSupplier`) — there is no `CharFunction` or `FloatPredicate`.

Code

```
// code/ch11/functional/BuiltInInterfaces.java (key parts)
Supplier<List<String>> newList = ArrayList::new;
Consumer<String> log = s -> System.out.println("log: " + s);
BiConsumer<String, Integer> put = (k, v) -> System.out.println(k + "=" + v);
Predicate<Order> big = o -> o.amount() > 100;
Function<Order, String> toId = Order::id;
BiFunction<Double, Double, Double> pow = Math::pow;
UnaryOperator<String> shout = s -> s.toUpperCase() + "!";
BinaryOperator<Integer> max = BinaryOperator.maxBy(Comparator.naturalOrder());

IntPredicate even = n -> n % 2 == 0;           IntUnaryOperator square = n -> n * n;
ToIntFunction<String> len = String::length;    IntFunction<String> stars = "*"::repeat;
ObjIntConsumer<String> repeatPrint = (s, n) -> System.out.println(s.repeat(n));

Function<Integer, Integer> boxedInc = x -> x + 1;   // boxes on every call
IntUnaryOperator primInc = x -> x + 1;              // no boxing
```

Output:

```
log: supplier made [x]
retries=3
big? true, longerThan("java",3)? true, id=A-1, 2^10=1024.0, HEY!, max=9
even(6)=true square(7)=49 add=5 len=5 stars=*** dice=4 ratio=0.25
ababab
20M calls: Function<Integer,Integer> 57 ms vs IntUnaryOperator 8 ms (machine-dependent) results equal=true
```

The last line is a rough single-run timing (Apple M4 Pro, JDK 26 — **machine-dependent**; the JIT — the JVM's *just-in-time* compiler, which turns frequently run bytecode into optimised machine code while the program runs — can sometimes remove boxing on its own). The point stands: in hot loops over numbers, pick the primitive interface.

When to write your own

- The shape doesn't exist (three parameters, or it throws a checked exception — §11.8).
- A domain name adds clarity and you'll attach default methods (`Discount.then` in §11.2).
- Annotate it with `@FunctionalInterface`.

Self-check

1. Which interface fits `list.removeIf(...)`? → `Predicate<? super E>`.

2. Which fits `map.merge(k, 1, ...)`? → `BiFunction<? super V, ? super V, ? extends V>` (commonly written `Integer::sum`).
3. What's the method of `ToIntFunction`? → `applyAsInt`.
4. `UnaryOperator<T>` extends which interface? → `Function<T, T>`.

TL;DR

Supplier (get), Consumer (accept), Predicate (test), Function (apply) plus `Bi`/`Unary`/`Binary` variants cover almost everything. Use `Int`/`Long`/`Double` versions for primitives to avoid boxing.

11.4 Method references

What it is

A **method reference** is shorthand for a lambda that does nothing but call one existing method: `String::length` means `s -> s.length()`. Like lambdas they need a target functional interface. *Effective Java* Item 43: prefer them when they are clearer than the lambda — and keep the lambda when it isn't.

The four kinds

Kind	Syntax	Equivalent lambda	Example
Static method	`Class::staticMethod`	`(args) -> Class.staticMethod(args)`	`Integer::parseInt`
Bound instance (specific object)	`obj::method`	`(args) -> obj.method(args)`	`System.out::println`, `prefix::concat`
Unbound instance (any object of a type)	`Class::instanceMethod`	`(obj, args) -> obj.method(args)`	`String::toUpperCase`, `String::startsWith`
Constructor	`Class::new`	`(args) -> new Class(args)`	`ArrayList::new`, `User::new`, `int[]::new`

The **unbound** form is the one that confuses people: the *first* parameter of the functional interface becomes the object the method is called on. So `BiFunction<String, String, Boolean> f = String::startsWith` means `(s, p) -> s.startsWith(p)`.

Code

```
// code/ch11/methodrefs/MethodRefDemo.java (key parts)
record User(String name, int age) {
    boolean isAdult() { return age >= 18; }
    static int byAge(User a, User b) { return Integer.compare(a.age, b.age); }
}
Function<String, Integer> parse = Integer::parseInt;          // static
Comparator<User> ageOrder = User::byAge;                      // static, shape (User, User) -> int
Function<String, String> addPrefix = prefix::concat;          // bound
Consumer<Object> printer = System.out::println;               // bound
Function<String, String> upper = String::toUpperCase;        // unbound
Predicate<User> adult = User::isAdult;                        // unbound
BiFunction<String, String, Boolean> startsWith = String::startsWith;   // unbound, 2 params
Supplier<List<User>> listMaker = ArrayList::new;              // constructor
BiFunction<String, Integer, User> makeUser = User::new;       // constructor with 2 args
IntFunction<int[]> arrayMaker = int[]::new;                   // array constructor

StringBuilder sb = new StringBuilder("v1");
Supplier<String> snap = sb::toString;                         // receiver sb fixed now
sb.append("+v2");
```

Output:

```
[User[name=Zed, age=17], User[name=Kim, age=25], User[name=Amy, age=34]]
43 ID-7 OK adult(Zed)=false startsWith=true array length=5
4 5
bound receiver is the same object: v1+v2
Amy Kim Zed
```

Notes:

- `parse.apply("42") + 1` printed `43`: the `Integer` result was unboxed and added before string concatenation started.
- `String::length` fits both `ToIntFunction<String>` and `Function<String, Integer>` — the target type decides.
- A **bound** reference evaluates its receiver expression **once**, when the reference is created (and throws `NullPointerException` — the error you get when using a reference that points to no object — right then if it's `null`). It holds that *object*, so later mutations of the object are visible (`v1+v2`), but reassigning the variable afterwards would not change the reference.
- Overloaded methods (several methods sharing one name but taking different parameters) can make a reference ambiguous (e.g. `Integer::toString` fits both the static `toString(int)` and the instance `toString()` for a `Function<Integer, String>`) — use a lambda then.

Self-check

1. Rewrite `s -> s.trim()` as a method reference. → `String::trim`.
2. Rewrite `() -> new HashMap<String, Integer>()`. → `HashMap::new` (target `Supplier<Map<String,Integer>>`).
3. What does `Comparator<String> c = String::compareToIgnoreCase;` mean? → `(a, b) -> a.compareToIgnoreCase(b)` — unbound, first parameter is the receiver.

TL;DR

Four kinds: `Class::static`, `obj::method` (bound), `Class::method` (unbound: first argument is the receiver), `Class::new`. Use when it reads better than the lambda.

11.5 Variable capture and "effectively final"

What it is

A lambda can use variables from the surrounding code — this is called **capturing**, and a function that carries captured variables is a **closure**. Java's rule: a captured **local variable or parameter** must be **final or effectively final** — never reassigned after initialisation (whether or not it says `final`).

Why the rule exists

1. Local variables live on the method's **stack frame** (the private scratch area a method call gets for its locals and parameters), which disappears when the method returns, but a lambda can outlive it (stored, passed to another **thread** — an independent path of execution running alongside others in the same program, Chapter 14). So Java **copies the value** into the lambda object.
2. If the variable could change later, the copy and the original would silently disagree. Forbidding reassignment makes the copy always correct.
3. It also prevents a whole class of **data races** (two threads touching the same variable at the same time, at least one writing, with no coordination — giving unpredictable results): a local variable mutated by lambdas running on several threads.

Fields are different. Instance fields are reached through the captured `this` reference (and static fields directly), so lambdas may read and modify them freely. That is legal — and exactly why it's easy to create thread-safety bugs (code that breaks only when several threads use it at once) with shared **mutable** (changeable) fields.

Code

```java
// code/ch11/capture/CaptureDemo.java (key parts)
int threshold = 10;                          // effectively final
Predicate<Integer> above = x -> x > threshold;
// threshold = 11;                           // would make the lambda above a compile error

for (String s : List.of("a", "b", "c")) suppliers.add(() -> s.toUpperCase());   // new 's' each iteration: fine
for (int i = 0; i < 3; i++) { int copy = i; byIndex.add(() -> copy * 10); }    // i is reassigned: copy it

AtomicInteger count = new AtomicInteger();                  // mutable holder: legal but a smell
List.of("x", "yy", "zzz").forEach(w -> count.addAndGet(w.length()));
int better = List.of("x", "yy", "zzz").stream().mapToInt(String::length).sum();   // return a result instead

Runnable lambda = () -> System.out.println("lambda this -> " + this.name);        // enclosing instance
Runnable anon = new Runnable() { public void run() { System.out.println(this.getClass().getName()); } };
Runnable inc = () -> counterField++;                        // fields may be modified
```

Output:

```
above(12)=true
ABC
0 10 20
holder count=6 vs stream sum=6
lambda this      -> CaptureDemo instance
anonymous this   -> ch11.capture.CaptureDemo$1
field modified from lambda: 2
```

Common mistakes

- Capturing the classic `for (int i…)` index — it's reassigned by `i++`. Copy to a local or use `IntStream.range(0, n).forEach(i -> …)`.
- Working around the rule with one-element arrays (`int[] c = {0}; list.forEach(x -> c[0]++)`) or `AtomicInteger` (a thread-safe integer holder whose updates happen as one indivisible step) just to accumulate. It compiles, but you've re-created shared mutable state; if the code later runs in parallel it breaks. Compute a result instead (`sum`, `count`, `collect`).
- Assuming a captured *object* is frozen — only the variable is effectively final; the object it points to can still be mutated.

Self-check

1. Can a lambda modify a captured local variable? → No; it can't even read one that is reassigned anywhere.
2. Can it modify an instance field? → Yes (via `this`), which is also why that can be unsafe across threads.
3. Why does capturing the variable of an enhanced for loop work? → Each iteration declares a fresh variable that is never reassigned.

TL;DR

Captured locals must be effectively final (values are copied into the lambda). Fields aren't restricted. Don't smuggle mutation through arrays or atomics — return results instead.

11.6 Composing functions

What it is

Composition builds a new function from existing ones, like joining pipes. The functional interfaces ship default and static methods for this, so you can assemble behaviour from small, tested, named pieces.

Interface	Composition methods
`Function`	`f.andThen(g)` = g(f(x)); `f.compose(g)` = f(g(x)); `Function.identity()`
`Predicate`	`and`, `or`, `negate`, `Predicate.not(p)` (11), `Predicate.isEqual(x)`
`Consumer` / `BiConsumer`	`andThen` — run one after the other
`BiFunction`	`andThen(Function)`
`BinaryOperator`	`minBy(comparator)`, `maxBy(comparator)`
`UnaryOperator`	`identity()`
`Comparator`	`comparing`, `thenComparing`, `reversed`, `nullsFirst` (§10.8)

Code

```
// code/ch11/composition/CompositionDemo.java (key parts)
Function<Integer, Integer> plus2 = x -> x + 2, times3 = x -> x * 3;
plus2.andThen(times3).apply(5);     // (5 + 2) * 3
plus2.compose(times3).apply(5);     // 5 * 3 + 2

UnaryOperator<String> trim = String::strip, lower = String::toLowerCase;
UnaryOperator<String> collapse = s -> s.replaceAll("\\s+", " ");
Function<String, String> clean = trim.andThen(lower).andThen(collapse);

Predicate<String> nonBlank = Predicate.not(String::isBlank);
Predicate<String> shortWord = s -> s.length() <= 4, startsJ = s -> s.startsWith("J");
filter(words, nonBlank.and(shortWord));   filter(words, startsJ.or(shortWord));
filter(words, startsJ.negate().and(nonBlank));   filter(words, Predicate.isEqual("Go"));

Consumer<String> audit = s -> System.out.print("[audit " + s + "] ");
audit.andThen(send).accept("msg-1");

Function<Double, UnaryOperator<Double>> taxAdder = rate -> price -> price * (1 + rate);   // function factory
UnaryOperator<Double> vat20 = taxAdder.apply(0.20);
```

Output:

```
plus2.andThen(times3)(5) = 21, plus2.compose(times3)(5) = 17
identity = same
clean -> [hello world]
nonBlank&short: [Java, Go, JVM]  startsJ|short: [Java, , Jakarta, Go,   , JVM]  !startsJ: [Go]  isEqual(Go): [Go]
[audit msg-1] [send msg-1]
area.andThen(format) = 12 m2, minBy length = fig
vat20(50.0) = 60.0
length, then reverse alpha: [a, cc, bb, aa, ddd]
```

Notes:

- `and` / `or` **short-circuit** (stop as soon as the answer is known) like `&&` / `||`: the second predicate isn't evaluated when the first decides.
- `startsJ.or(shortWord)` kept `""` and `" "` because they are short — combine with `nonBlank` when that matters.
- `Predicate.not(String::isBlank)` exists because you can't write `String::isBlank.negate()` — a method reference has no type until it's assigned.
- `trim.andThen(lower)` returns a `Function`, not a `UnaryOperator` (the default method is inherited from `Function`).

Real-world examples

Validation rules (`Predicate<Order>` combined per market), request-processing **middleware** (code that sits between an incoming request and the handler, e.g. adding logging or auth; here a chain of `UnaryOperator<Request>`), text-normalisation pipelines in search indexing (preparing text so it can be searched quickly), configurable pricing rules (function factories).

Self-check

1. `f.andThen(g)` vs `f.compose(g)`? → `andThen` runs f first; `compose` runs g first.
2. How to keep strings that are *not* empty using a method reference? → `Predicate.not(String::isEmpty)`.
3. What does `Function.identity()` return? → A function that returns its input unchanged (handy as a key/value mapper, e.g. `toMap(identity(), …)`).

TL;DR

Build big behaviour from small functions: `andThen`/`compose` for functions, `and`/`or`/`negate`/`not` for predicates, `andThen` for consumers, and functions that return functions for configurable logic.

11.7 `Optional`

What it is

`Optional<T>` (Java 8) is a container that holds either one non-null value or nothing. It is meant for **method return types** where "no result" is a normal outcome — `findById`, `max`, `findFirst` — so the signature itself tells the caller to handle absence. Analogy: a parcel delivery that may arrive empty; the box forces you to look inside before using the contents, whereas a `null` is a missing box you trip over.

Why it matters

`null` returned from a method says nothing in the signature, and forgetting a check causes a `NullPointerException` far from the cause. `Optional` makes absence explicit and gives fluent operations to handle it without nested `if (x != null)` chains.

The API

Group	Methods
Create	`Optional.of(v)` (NPE if `v` is null), `Optional.ofNullable(v)`, `Optional.empty()`
Test	`isPresent()`, `isEmpty()` (11)
Transform	`map(f)` (wraps the result; a `null` result becomes empty), `flatMap(f)` (when `f` already returns an `Optional`), `filter(p)`
Extract	`orElse(default)`, `orElseGet(supplier)` (a supplier is a no-argument function that produces a value), `orElseThrow()` (10), `orElseThrow(exceptionSupplier)`, `get()` (throws if empty)
Act	`ifPresent(consumer)`, `ifPresentOrElse(consumer, runnable)` (9)
Combine	`or(() -> otherOptional)` (9), `stream()` (9) → 0 or 1 elements
Primitive versions	`OptionalInt`, `OptionalLong`, `OptionalDouble` (`getAsInt()` …)

Code

```
// code/ch11/optional/OptionalDemo.java (key parts)
record Address(String city, String zip) {}
record Customer(String name, Address address, String email) {
    Optional<Address> addressOpt() { return Optional.ofNullable(address); }
    Optional<String> emailOpt() { return Optional.ofNullable(email); }
}
static Optional<Customer> findById(int id) { return Optional.ofNullable(DB.get(id)); }

String city = findById(1).flatMap(Customer::addressOpt).map(Address::city).orElse("no city");

findById(1).map(Customer::name).orElse(expensiveDefault());            // default computed ANYWAY
findById(1).map(Customer::name).orElseGet(OptionalDemo::expensiveDefault); // computed only if empty

findById(9).orElseThrow(() -> new IllegalArgumentException("customer 9 not found"));
findById(2).flatMap(Customer::emailOpt).ifPresentOrElse(e -> ..., () -> System.out.println("Bo has no email ->
send SMS"));
findById(2).flatMap(Customer::emailOpt).or(() -> Optional.of("sms:+000"));
List.of(1, 2, 9).stream().map(OptionalDemo::findById).flatMap(Optional::stream)...   // drop empties
```

Output:

```
Optional[hi] Optional.empty Optional.empty isPresent=true isEmpty=true
Optional.of(null) -> NPE
cities: Lisbon, no city, no city
gmail? false
orElse on present:    (computing default) Ana
orElseGet on present: Ana
orElseThrow() -> No value present
orElseThrow(supplier) -> customer 9 not found
Bo has no email -> send SMS
fallback Optional via or(): sms:+000
all known emails: [ana@example.com]
OptionalInt max=3, empty average=OptionalDouble.empty
anti-pattern isPresent+get: Ana
better: Ana
```

Read the chain `findById(id).flatMap(Customer::addressOpt).map(Address::city).orElse("no city")` as: find the customer → if found, get their address (which might itself be missing, so `flatMap` to avoid `Optional<Optional<Address>>`) → if present, take the city → otherwise "no city". Three possible "nothing" points, zero `if` statements.

The `orElse` line proves that **`orElse`'s argument is always evaluated** — it's an ordinary method argument. Use `orElseGet` when the default is expensive or has **side effects** (anything beyond returning a value — creating objects, DB calls, logging; defined fully in §11.8).

Right uses and wrong uses

✅ Do	❌ Don't
Return `Optional<T>` from a method when "no result" is normal	Return `null` from a method declared to return `Optional`
Chain `map` / `flatMap` / `filter` and finish with `orElse*` / `orElseThrow` / `ifPresent*`	`if (opt.isPresent()) opt.get()` — a verbose null check
Use `orElseThrow()` (10) when absence is a bug — its name says it can throw	Call `get()` without a check (`orElseThrow()` is the same thing with an honest name)
Use `OptionalInt` / `OptionalDouble` for primitives	`Optional<Integer>` in numeric code
Return an **empty collection** for "no items"	`Optional<List<T>>` — an empty list already means "none"
Use nullable fields internally and expose `Optional` from getters if helpful	`Optional` **fields** (not `Serializable` — can't be turned into bytes by Java's built-in serialisation — extra object per field)
Overload or use a default for optional inputs	`Optional` **parameters** (callers must wrap; they can still pass `null`)
Use it as a return value	Use it as a map key/value or in collections

These guidelines follow *Effective Java* Item 55 ("return optionals judiciously") and the intent stated by the JDK designers: `Optional` is primarily for return types. It is also a *value-based* class (one whose instances should be treated purely as values, with no meaningful identity) — don't compare instances with `==` or synchronise on them (use them as a lock, Chapter 14).

In the interview

> **30-second answer:** "Optional is a return-type container for 'maybe a value'. It makes absence explicit in the signature and lets me chain map, flatMap and filter and end with orElse, orElseGet or orElseThrow instead of null checks. I don't use it for fields, parameters or collections, I never return null from an Optional method, and I use orElseGet when the default is expensive because orElse evaluates its argument eagerly."

- *Follow-up:* `map` *vs* `flatMap`*?* → Use `flatMap` when the function itself returns an `Optional`, to avoid nesting.
- *Follow-up:* `Optional.of` *vs* `ofNullable`*?* → `of` throws NPE on null (use when null is a bug); `ofNullable` returns empty.
- *Follow-up: Is* `Optional` *serializable?* → No — one reason not to use it for fields.

Self-check

1. What does `Optional.ofNullable(null).map(String::length)` return? → `Optional.empty`.
2. Which is lazy: `orElse` or `orElseGet`? → `orElseGet`.
3. How do you turn a `Stream<Optional<T>>` into a `Stream<T>` of present values? → `.flatMap(Optional::stream)` (Java 9).

TL;DR

`Optional` = explicit "maybe" for return values. Transform with `map` / `flatMap` / `filter`, end with `orElse` / `orElseGet` / `orElseThrow` / `ifPresentOrElse`. Not for fields, parameters or collections; never `null` itself.

11.8 Functional style in everyday Java

What it is

Functional programming (FP) builds programs from **functions** that compute results from inputs, avoiding shared mutable state. Java is not a pure functional language, but several FP ideas make Java code simpler and safer. Key terms:

Term	Meaning
Pure function	Result depends only on its arguments; no side effects (no I/O, no mutation of outside state). Same input → same output, every time.
Side effect	Anything a function does besides returning a value: printing, writing a field, changing an argument, I/O.
Immutability	Data that can't change after creation (records, `List.of`, `String`).
Higher-order function	Takes a function as a parameter and/or returns one (`retryUntil`, `memoize`, `Comparator.comparing`).
Currying	Turning `f(a, b)` into `a -> b -> …`, so you can fix `a` first (**partial application**).
Laziness	Delaying a computation until its result is needed (`Supplier`, streams in Chapter 12).
Memoisation	Caching a pure function's results by input.
Declarative	Describe *what* result you want (`filter`, `map`) rather than *how* to loop — the style Chapter 12 is built on.

Code

```
// code/ch11/fp/FunctionalStyle.java (key parts)
static int total = 0;
static int addToTotal(int x) { total += x; return total; }    // impure: result depends on history
static int add(int a, int b) { return a + b; }                // pure

static <T> List<T> retryUntil(Supplier<T> attempt, Predicate<T> ok, int max) { ... }   // higher-order

Function<Integer, Function<Integer, Integer>> curriedAdd = a -> b -> a + b;
Function<Integer, Integer> add10 = curriedAdd.apply(10);      // partial application

static void logIf(boolean enabled, Supplier<String> message) { if (enabled) System.out.println(message.get()); }
logIf(debug, () -> "state dump " + "x".repeat(1_000_000).length());   // string never built when debug=false

static <T, R> Function<T, R> memoize(Function<T, R> f) {
    Map<T, R> cache = new HashMap<>();
    return x -> { R r = cache.get(x); if (r == null) { r = f.apply(x); cache.put(x, r); } return r; };
}

Map<String, DoubleUnaryOperator> shipping = Map.of(           // strategy pattern as data
        "standard", w -> 4.99, "express", w -> 9.99 + 1.5 * w, "free", w -> 0.0);

@FunctionalInterface interface ThrowingFunction<T, R> { R apply(T t) throws Exception; }
static <T, R> Function<T, R> unchecked(ThrowingFunction<T, R> f) {
    return t -> {
        try { return f.apply(t); }
        catch (RuntimeException e) { throw e; }
        catch (Exception e) { throw new RuntimeException(e); }
    };
}
```

Output:

```
impure twice: 5, 10   pure twice: 5, 5
retryUntil six: [2, 5, 6]
add10(5)=15
lazy log: supplier not called because debug=false
memoized: 5 calls, underlying function ran 2 times
standard(2kg)=4.99  express(2kg)=12.99  free(2kg)=0.00
app.properties=ok
wrapped: java.io.IOException: no such file: missing.properties
original [10, 20, 30] untouched; new [9, 18, 27]
```

What each idea buys you:

- **Pure functions** are trivially testable (no setup, no mocks), safe to cache and safe to run in parallel.
- **Higher-order functions** remove duplicated control flow: the retry loop is written once; callers pass *what* to try and *when* it succeeded.
- **Laziness via** `Supplier` is how logging frameworks avoid building expensive messages (SLF4J — the Simple Logging Facade for Java, a common logging front end — and its fluent API and `java.util.logging.Logger.fine(Supplier<String>)` both accept suppliers).
- **Strategies as lambdas** replace a hierarchy of one-method classes; see the Strategy pattern in LLD Chapter 10.
- **Immutability**: transform into a new list instead of editing in place; the original stays valid for anyone else holding it.

Memoisation note. The obvious `cache.computeIfAbsent(x, f)` is wrong when `f` is *recursive* (e.g. memoised Fibonacci calls back into the same map): `HashMap.computeIfAbsent` must not be modified by its own mapping function and throws `ConcurrentModificationException` (since Java 9) — `ConcurrentHashMap` (the thread-safe map) can **deadlock** (get stuck forever waiting on itself) or throw `IllegalStateException`. The get-then-put version above is safe for single-threaded use.

Checked exceptions (exceptions the compiler forces you to either catch or declare with `throws`, such as `IOException`). `java.util.function` methods don't declare `throws`, so a lambda calling `Files.readString` (which throws `IOException`) won't compile as a `Function`. Options: handle it inside the lambda, wrap it as above (rethrow as unchecked — `UncheckedIOException` is the JDK's own choice for I/O), or define your own throwing interface. Don't silently swallow the exception. See Chapter 8 for checked vs unchecked.

When not to use functional style

- The loop is simpler: a `for` with early `return` or several interacting variables is often clearer than a clever chain.
- Deeply nested lambdas (a function returning a function returning a function) — name intermediate pieces.
- Performance-critical inner loops where boxing or **megamorphic** calls (one call site that sees many different implementing classes, which stops the JIT from inlining it) matter — measure (Chapter 16).
- Stack traces through lambdas include **synthetic** frames (methods the compiler generated, not ones you wrote) like `lambda$main$0`; debugging a long chain is harder than a named method.

Real-world examples

Spring's (the most popular Java application framework) `RestClient`/`WebClient` builders and Security configuration **DSLs** (domain-specific languages: mini-APIs designed to read like a small language for one job) take lambdas; `CompletableFuture` chains functions over **async** (asynchronous: finishing later, without blocking the caller) results (Chapter 15); event listeners and callbacks in UI frameworks; retry/backoff helpers (backoff = waiting a bit longer between each retry; Resilience4j, a fault-tolerance library, decorates `Supplier`s and `Function`s); JUnit's (the standard Java unit-testing library, Chapter 18) `assertThrows(Class, Executable)`.

In the interview

> **30-second answer:** "In Java I use functional ideas where they reduce bugs: pure functions and immutable data for anything shared or tested, higher-order functions to remove repeated control flow, Suppliers for laziness, and lambdas as strategies. I stop when a plain loop is clearer, and I'm careful with checked exceptions and mutable captured state."

- *Follow-up: Is Java a functional language?* → No — it's object-oriented with functional features (first-class-ish functions — "first-class" meaning they can be stored, passed and returned like any other value — via functional interfaces, lambdas, streams, records, pattern matching). Functions are objects implementing interfaces, not a separate function type.
- *Follow-up: What's a pure function and why care?* → Output depends only on input, no side effects; easy to test, cache, and parallelise.

Self-check

1. Is `x -> { System.out.println(x); return x; }` pure? → No; printing is a side effect.
2. What's partial application of `a -> b -> a * b` with `a = 3`? → A function `b -> 3 * b`.
3. Why can't a lambda that calls `Files.readString` be a `Function<Path, String>` directly? → `IOException` is checked and `Function.apply` doesn't declare it.

TL;DR

Pure functions + immutable data + higher-order functions = code that's easier to test and parallelise. Use `Supplier` for laziness, lambdas for strategies, wrap checked exceptions deliberately, and fall back to loops when they're clearer.

Chapter TL;DR

Topic	Remember
Why	Pass behaviour as a value; lambdas replace anonymous classes (EJ Item 42)
Functional interface	Exactly one abstract method; `@FunctionalInterface`; lambda's type comes from the target
Syntax	`x -> expr`, `(a, b) -> { …; return r; }`, `(var a, var b)` (11), `_` for unused (22)
Built-ins	`Supplier.get`, `Consumer.accept`, `Predicate.test`, `Function.apply`, `Bi*`, `UnaryOperator`, `BinaryOperator`; `IntX`/`ToIntX` to avoid boxing
Method refs	`Class::static`, `obj::method`, `Class::method` (first arg is receiver), `Class::new`
Capture	Locals must be effectively final (value copied); fields free; `this` = enclosing instance
Composition	`andThen`/`compose`/`identity`, `and`/`or`/`negate`/`Predicate.not`, functions returning functions
Optional	Return type only; `map`/`flatMap`/`filter` → `orElse`/`orElseGet` (lazy)/`orElseThrow`; no fields/params/collections; never null
FP style	Pure, immutable, higher-order, lazy; wrap checked exceptions; loops when clearer

Where next: put all of this to work on collections with Chapter 12 — Streams. Exam traps: OCP Chapter 8. Python equivalents (lambdas, closures, `functools`): Python Chapter 7.